

Analyzing the Role of AI-Assisted Programming in Modern Software Development

Field	Details
Module	Advanced Programming Concepts
Assignment	Assessment 2 - AI-Assisted Programming
Selected System	Library Management System
Prepared by	Student Name:Desaa T.K.I
GitHub Repository	https://github.com/Kavi-Cs/ai-assisted-programming-library-system

Task 1: Conceptual Understanding

What Is AI-Assisted Programming?

AI-assisted programming is the use of artificial intelligence tools to support software development activities such as writing code, explaining logic, detecting defects, generating tests, refactoring, documenting programs, and suggesting design improvements. It does not replace the programmer's responsibility. Instead, it acts as a productivity assistant that can propose solutions while the developer reviews, tests, and adapts the output.

Types of AI Coding Tools

- Code generation tools create functions, classes, boilerplate, user interfaces, and test cases from natural language prompts.
- Debugging assistants explain error messages, trace possible causes, and suggest corrections.
- Refactoring assistants improve structure, naming, modularity, duplication, and maintainability.
- Documentation assistants generate comments, README files, API explanations, and user guides.
- Testing assistants propose unit tests, boundary cases, and sample input/output scenarios.

Advantages in Software Development

- Faster prototyping and reduced time spent on repetitive code.
- Improved learning support because students can ask for explanations and examples.
- Better exploration of alternative designs and implementation strategies.

- Assistance with testing, validation, and documentation.
- Useful support for unfamiliar libraries, syntax, and error messages.

Risks and Limitations

- AI may hallucinate methods, libraries, or logic that looks correct but fails during execution.
- Generated code can include security weaknesses, poor validation, or unsafe dependency choices.
- Students may become dependent on suggestions without understanding the underlying concepts.
- AI-generated output can raise plagiarism, ownership, and academic integrity concerns.
- The developer remains accountable for testing, correctness, licensing, and maintainability.

Task 2: Practical Implementation

The selected problem is a Library Management System for a small library. The system manages books, members, borrowing, returning, searching, and reporting of borrowing status. This problem is moderate in complexity because it requires object-oriented modelling, data validation, state changes, and interaction between multiple classes.

System Requirements

- Add books with book ID, title, author, and availability status.
- Register members with member ID, name, and borrowed book records.
- Borrow a book only if the member and book exist and the book is available.
- Return a book only if it was borrowed by that member.
- Display books, members, and borrowing status.
- Version B adds persistent JSON storage and search functionality.

Use Case Diagram - Library Management System

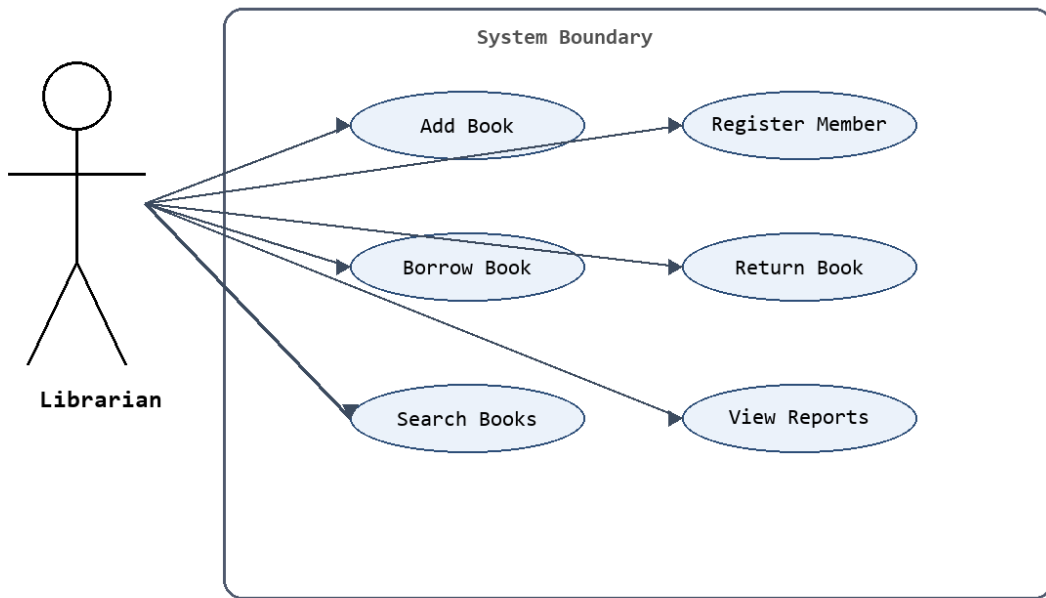


Figure 1: Use Case Diagram

Class Diagram - Library Management System

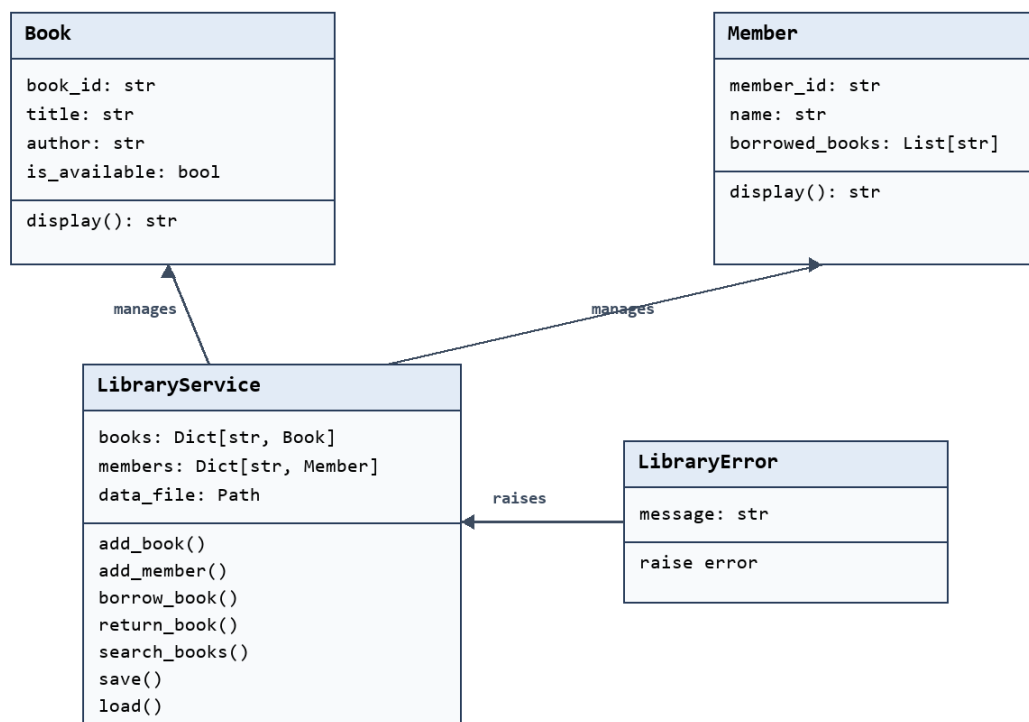


Figure 2: Class Diagram

Activity Diagram - Borrow Book

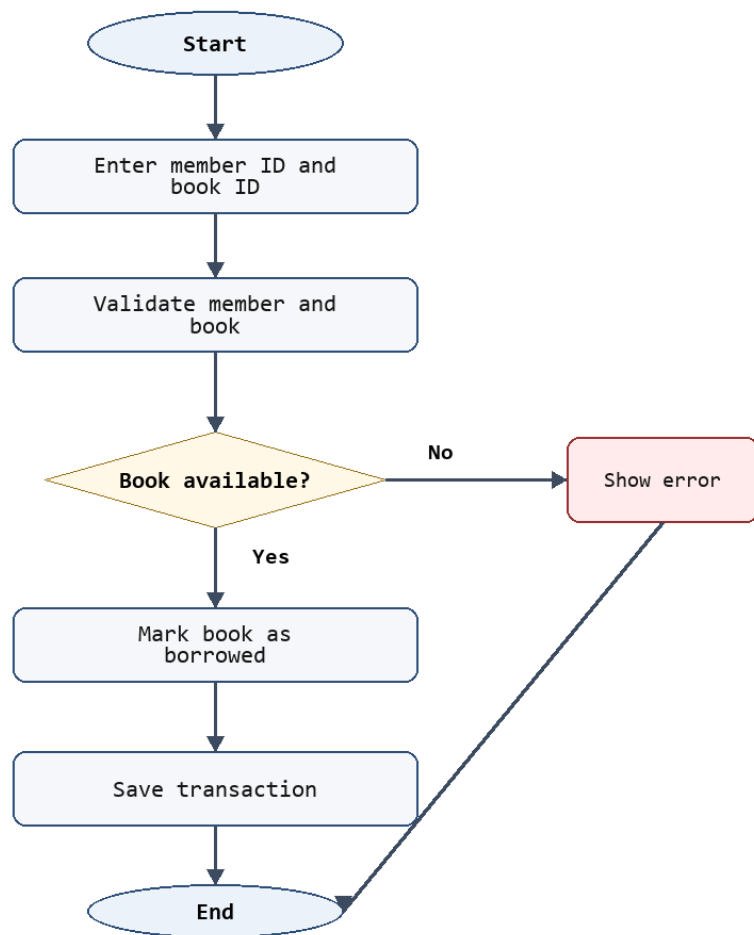


Figure 3: Activity Diagram for Borrow Book

Sequence Diagram - Borrow Book

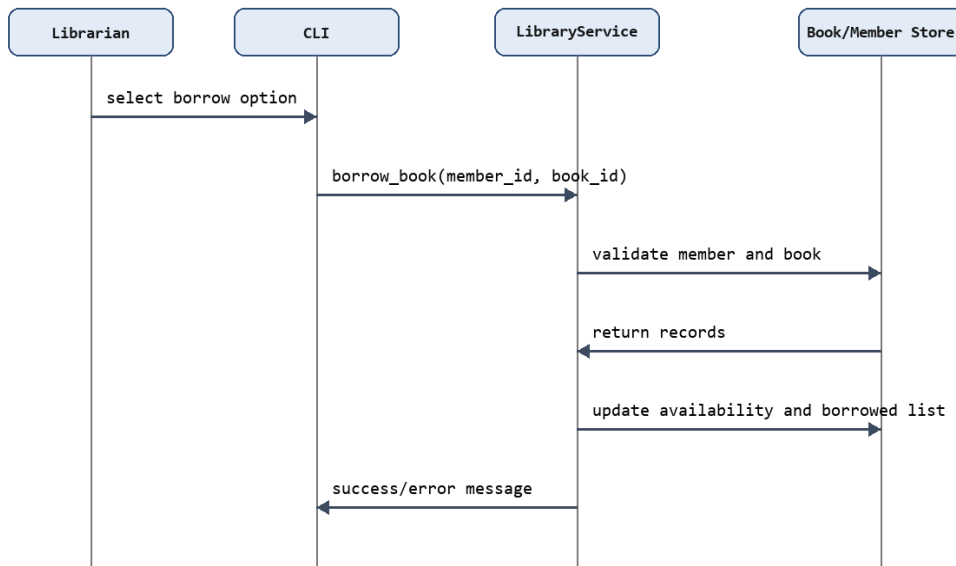


Figure 4: Sequence Diagram for Borrow Book

Version A: Traditional Programming Approach

Version A was designed as a traditional baseline. It uses basic Python classes, dictionaries, menu-driven input, and simple conditional checks. The program is easy to follow and demonstrates the essential object-oriented model, but it does not persist data after the program exits.

```
class Library:
    def add_book(self, book_id, title, author):
        if book_id in self.books:
            return "Book ID already exists."
        self.books[book_id] = Book(book_id, title, author)
        return "Book added successfully."
```

```
Library Management System - Version A
1. Add book
2. Add member
3. Borrow book
4. Return book
5. List books
6. List members
0. Exit
Select option: B001 | Clean Code | Robert C. Martin | Available
B002 | Python Crash Course | Eric Matthes | Available

Library Management System - Version A
1. Add book
2. Add member
3. Borrow book
4. Return book
5. List books
6. List members
0. Exit
Select option: Member ID: Book ID: Book borrowed successfully.

Library Management System - Version A
1. Add book
2. Add member
3. Borrow book
4. Return book
5. List books
6. List members
0. Exit
Select option: B001 | Clean Code | Robert C. Martin | Borrowed
B002 | Python Crash Course | Eric Matthes | Available

Library Management System - Version A
1. Add book
2. Add member
3. Borrow book
4. Return book
5. List books
6. List members
0. Exit
Select option: Exiting system.
```

Figure 5: Version A sample execution

Version B: AI-Assisted Programming Approach

Version B was developed using AI assistance to refine the original design. The improved implementation uses dataclasses, type hints, a custom exception class, JSON persistence, sorted output, input validation, and search. AI was most useful for proposing improvements and identifying missing quality features such as persistence and centralized error handling.

```
@dataclass
class Book:
    book_id: str
    title: str
    author: str
    is_available: bool = True
```

Library Management System - Version B

1. Add book
2. Add member
3. Borrow book
4. Return book
5. Search books
6. List books
7. List members
0. Exit

Select option: ID Title Author Status

B001	Clean Code	Robert C. Martin	Available
B002	Python Crash Course	Eric Matthes	Available
B003	Design Patterns	Gamma et al.	Available

Library Management System - Version B

1. Add book
2. Add member
3. Borrow book
4. Return book
5. Search books
6. List books
7. List members
0. Exit

Select option: Member ID: Book ID: Book borrowed and saved.

Library Management System - Version B

1. Add book
2. Add member
3. Borrow book
4. Return book
5. Search books
6. List books
7. List members
0. Exit

Select option: Keyword: ID Title Author Status

B001	Clean Code	Robert C. Martin	Borrowed
------	------------	------------------	----------

Library Management System - Version B

1. Add book
2. Add member
3. Borrow book
4. Return book
5. Search books
6. List books
7. List members
0. Exit

Select option: ID Name Borrowing Details

M001	Nimal Perera	Borrowed: B001
M002	Asha Silva	Borrowed: None

Library Management System - Version B

1. Add book
2. Add member
3. Borrow book
4. Return book
5. Search books
6. List books
7. List members
0. Exit

Select option: Exiting system.

Figure 6: Version B sample execution

Manual and AI-Assisted Code Identification

Code Area	Version A	Version B
Core classes	Manual-style baseline: Book, Member, Library	AI-assisted refinement: Book, Member, LibraryService
Data storage	In-memory dictionaries	AI-assisted JSON persistence using save/load methods
Error handling	Return message strings	AI-assisted custom LibraryError exceptions
User interface	Basic menu and direct input	Cleaner menu, validation, formatted output
Extra features	List books and members	Search, sorting, persistence, stronger checks

AI Prompts Used During Development

- Suggest a moderate-complexity programming problem suitable for comparing traditional programming and AI-assisted programming in an Advanced Programming Concepts assignment.
- Create a simple object-oriented Python Library Management System with books, members, borrowing, returning, and listing features.
- Improve this library management program using dataclasses, JSON persistence, cleaner error handling, validation, and a searchable book catalogue.
- Generate UML use case, class, activity, and sequence diagrams for a Library Management System that supports adding books, adding members, borrowing books, returning books, and searching books.
- Compare the traditional and AI-assisted implementations based on development time, code quality, error handling, readability, and maintainability.
- Explain ethical considerations of AI-generated code, including hallucinated code, security risks, over-dependence, plagiarism, and accountability.

Implementation Comparison

Criterion	Version A: Traditional	Version B: AI-Assisted
Development time	Approx. 2 hours because structure, validation, and menu logic were written step by step.	Approx. 1 hour because AI accelerated boilerplate, validation ideas, and refactoring.
Code quality	Simple and understandable, but repeated checks and message strings are spread across methods.	More modular and professional with type hints, dataclasses, persistence, and a service layer.

Error handling	Basic string responses; valid for simple use but less scalable.	Centralized exception handling gives cleaner user feedback and easier maintenance.
Readability	Very easy for beginners to read.	Slightly more advanced, but clearer for larger systems because responsibilities are separated.
Maintainability	Suitable for small demos; adding storage or search requires more changes.	Easier to extend because storage, validation, and operations are organized in one service class.

Task 3: Critical Analysis

How AI Changed the Coding Approach

AI changed the approach from writing only the minimum working program to thinking about quality improvements earlier. The traditional implementation focused on making the menu and core operations work. The AI-assisted implementation encouraged additional concerns such as persistence, type hints, better validation, clearer error messages, and search functionality.

Effect on Understanding

AI improved understanding when it was used as an explanation and review tool. It helped identify why dataclasses reduce repetitive constructor code and why a custom exception improves control flow. However, understanding would be reduced if the student copied the final answer without reading, testing, and modifying the code.

Cases Where AI Should Not Be Used

- When the developer does not have enough knowledge to verify the generated solution.
- When confidential data, private keys, or sensitive client code would be exposed to an external tool.
- When a safety-critical or security-critical system requires strict human review and certification.
- When academic rules prohibit AI-generated work or require individual unaided coding.
- When licensing or copyright status of generated code is unclear.

Ethical Considerations

Ethical use of AI in programming requires transparency, review, and accountability. Students should disclose prompts and AI-assisted sections, avoid submitting work they do not

understand, and test all generated code. Developers must also check for insecure logic, biased assumptions, hidden plagiarism, and unsuitable dependencies. AI can support learning, but it should not replace the learning process.

Task 4: Conclusion

AI-assisted programming is valuable in both education and industry when it is used responsibly. In education, it can act as a tutor that explains concepts and suggests improvements. In industry, it can speed up prototyping, documentation, testing, and refactoring. The comparison shows that AI assistance improved the Library Management System by adding persistence, validation, and maintainability. However, AI output must always be reviewed, tested, and understood by the programmer. The best approach is not to replace programmers with AI, but to use AI as a supervised assistant that supports stronger human problem-solving.

Repository Link

GitHub repository submission link: https://github.com/YOUR_USERNAME/ai-assisted-programming-library-system